

MATEMATICĂ

1. (Progresii geometrice)

Determinați $x \in \mathbb{R}$ astfel încât numerele $x + 1$, $x + 3$, $x + 7$ să fie termeni consecutivi ai unei progresii geometrice.

- A) 0 C) 2 E) 3
 B) 1 D) -1 F) nu există

Răspuns corect: B

Rezolvare

Trei numere sunt termeni consecutivi ai unei progresii geometrice dacă și numai dacă pătratul termenului din mijloc este egal cu produsul vecinilor:

$$(x + 3)^2 = (x + 1)(x + 7)$$

$$x^2 + 6x + 9 = x^2 + 8x + 7 \Rightarrow 2x = 2 \Rightarrow x = 1.$$

Verificare: termenii devin 2, 4, 8 — progresie geometrică cu rația $q = 2$. ✓

2. (Determinant circulant și Relațiile lui Viète)

Fie x_1, x_2, x_3 rădăcinile polinomului $f = X^3 - 2X^2 + X - 3$. Calculați:

$$\Delta = \begin{vmatrix} x_1 & x_2 & x_3 \\ x_2 & x_3 & x_1 \\ x_3 & x_1 & x_2 \end{vmatrix}$$

- A) 2 C) -2 E) -1
 B) 0 D) 3 F) 1

Răspuns corect: C

Rezolvare

Determinantul se descompune astfel: $\Delta = -(x_1 + x_2 + x_3) \left(\sum x_i^2 - \sum_{i < j} x_i x_j \right)$.

Din relațiile lui Viète avem: $S_1 = x_1 + x_2 + x_3 = 2$ și $S_2 = x_1 x_2 + x_2 x_3 + x_3 x_1 = 1$.

A doua paranteză devine: $\sum x_i^2 - \sum x_i x_j = S_1^2 - 3S_2 = 2^2 - 3 \cdot 1 = 1$.

Deci $\Delta = -(2) \cdot (1) = -2$.

3. (Numere complexe – modul și conjugat)

Fie $z \in \mathbb{C}$ cu $|z| = 2$. Calculați:

$$|z + \bar{z}|^2 + |z - \bar{z}|^2$$

- A) 4 C) 16 E) 32
 B) 8 D) 2 F) 12

Răspuns corect: C

Rezolvare

Scriem $z = a + bi$, cu $a, b \in \mathbb{R}$. Atunci $\bar{z} = a - bi$, deci:

$z + \bar{z} = 2a$ și $z - \bar{z} = 2bi$.

Rezultă: $|z + \bar{z}|^2 + |z - \bar{z}|^2 = 4a^2 + 4b^2 = 4(a^2 + b^2) = 4|z|^2 = 4 \cdot 4 = 16$.

Observație: identitatea $|z + \bar{z}|^2 + |z - \bar{z}|^2 = 4|z|^2$ este un caz particular al identității paralelogramului — rezultatul nu depinde de argumentul lui z , ci doar de modul.

4. (Integrala pe interval simetric (paritate))

Calculați:

$$I = \int_{-1}^1 \frac{x^{2025} + \sin(x^3) + 1}{x^2 + 1} dx$$

- A) 0 C) $\frac{\pi}{2}$ E) 1
 B) π D) $\frac{\pi}{4}$ F) 2025

Răspuns corect: C

Rezolvare

Separăm integrala: $I = \int_{-1}^1 \frac{x^{2025} + \sin(x^3)}{x^2 + 1} dx + \int_{-1}^1 \frac{1}{x^2 + 1} dx$.

Deoarece funcția $f(x) = \frac{x^{2025} + \sin(x^3)}{x^2 + 1}$ este impară ($f(-x) = -f(x)$), integrala ei pe intervalul simetric $[-1, 1]$ este 0.

Rămâne integrala părții pare: $\int_{-1}^1 \frac{1}{x^2 + 1} dx = \arctan x \Big|_{-1}^1 = \frac{\pi}{4} - \left(-\frac{\pi}{4}\right) = \frac{\pi}{2}$.

5. (Sistem liniar cu parametru)

Determinați valorile parametrului $m \in \mathbb{R}$ pentru care sistemul de mai jos este compatibil nedeterminat:

$$\begin{cases} x + y + z = 1 \\ x + 2y + 3z = 2 \\ x + 3y + mz = 3 \end{cases}$$

- A) $m = 5$ C) $m \neq 5$ E) orice m
 B) $m = 3$ D) $m = 0$ F) niciun m

Răspuns corect: A

Rezolvare

Calculăm determinantul matricei sistemului:

$$\det A = \begin{vmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 1 & 3 & m \end{vmatrix} = (2m - 9) - (m - 3) + (3 - 2) = m - 5.$$

Pentru $m \neq 5$: $\det A \neq 0$, sistem compatibil **determinat** (soluție unică) — nu convine.
 Pentru $m = 5$: observăm că $L_3 = 2L_2 - L_1$ atât în stânga $((1, 3, 5) = 2(1, 2, 3) - (1, 1, 1))$, cât și în dreapta $(3 = 2 \cdot 2 - 1)$.
 A treia ecuație este deci o consecință a primelor două: rangul matricei = rangul matricei extinse = $2 < 3$ necunoscute $\Rightarrow$ sistem compatibil **nedeterminat**.
 Așadar $m = 5$.

6. (Studiul funcțiilor – valoare maximă)

Fie funcția $f : (0, \infty) \rightarrow \mathbb{R}$, $f(x) = \frac{\ln x}{x}$. Valoarea maximă a funcției este:

A) e C) $\frac{1}{e}$ E) $\frac{1}{e^2}$ B) 1 D) 0 F) $\frac{2}{e}$

Răspuns corect: C

Rezolvare

Calculăm derivata cu regula câtului: $f'(x) = \frac{\frac{1}{x} \cdot x - \ln x \cdot 1}{x^2} = \frac{1 - \ln x}{x^2}$.

Semnul derivatei este dat de numărător: $1 - \ln x > 0 \Leftrightarrow x < e$.

Deci f este strict crescătoare pe $(0, e)$ și strict descrescătoare pe $(e, \infty) \Rightarrow x = e$ este punct de maxim global.

Valoarea maximă: $f(e) = \frac{\ln e}{e} = \frac{1}{e}$.

Bonus de cultură matematică: din acest rezultat decurge celebra inegalitate $e^\pi > \pi^e$.

7. (Tangenta dusă dintr-un punct exterior)

Tangenta la graficul funcției $f(x) = e^x$ care trece prin originea $O(0, 0)$ are panta:

A) 1 C) $\frac{1}{e}$ E) 0 B) e D) e^2 F) $2e$

Răspuns corect: B

Rezolvare

Fie a abscisa punctului de tangență. Ecuația tangentei în punctul (a, e^a) este:

$y - e^a = e^a(x - a)$, deoarece $f'(x) = e^x$.

Impunem condiția ca tangenta să treacă prin origine $(x = 0, y = 0)$:

$-e^a = e^a(-a) \Rightarrow e^a(a - 1) = 0$.

Cum $e^a \neq 0$, rezultă $a = 1$. Punctul de tangență este $(1, e)$, iar panta este $f'(1) = e$.

Verificare: dreapta $y = ex$ trece prin O și prin $(1, e)$. ✓

8. (Manipulare algebrică - suma de pătrate)

Rezolvați, pentru $x \geq 1, y \geq 2, z \geq 3$:

$$\sqrt{x-1} + \sqrt{y-2} + \sqrt{z-3} = \frac{x+y+z-3}{2}$$

Calculați $x + y + z$.

A) 6

C) 14

E) 0

B) 9

D) 12

F) o infinitate

Răspuns corect: B

Rezolvare

Înmulțim ecuația cu 2 și mutăm totul într-o parte pentru a forma pătrate perfecte:

$$(x-1-2\sqrt{x-1}+1) + (y-2-2\sqrt{y-2}+1) + (z-3-2\sqrt{z-3}+1) = 0$$

$$(\sqrt{x-1}-1)^2 + (\sqrt{y-2}-1)^2 + (\sqrt{z-3}-1)^2 = 0.$$

Fiecare pătrat trebuie să fie 0 $\Rightarrow x=2, y=3, z=4$, deci suma este $x+y+z=9$.

9. (Ecuație trigonometrică – numărare grafică)

Numărul soluțiilor reale ale ecuației:

$$\sin x = \frac{x}{100}$$

A) 63

C) 31

E) 100

B) 62

D) 64

F) o infinitate

Răspuns corect: A

Rezolvare

Cum $|\sin x| \leq 1$, orice soluție verifică $|\frac{x}{100}| \leq 1$, adică $x \in [-100, 100]$.

Interpretăm grafic: numărăm intersecțiile dreptei $y = \frac{x}{100}$ (pantă foarte mică) cu sinusoida.

Pentru $x > 0$, intersecțiile apar doar pe “cocoașele” pozitive ale sinusoidelor, adică pe intervalele $(2k\pi, (2k+1)\pi)$. Cum $\frac{100}{\pi} \approx 31,8$, ultima cocoașă completă din $[0, 100]$ este $(30\pi, 31\pi)$, cu $31\pi \approx 97,4 < 100$, deci $k \in \{0, 1, \dots, 15\}$.

Pe fiecare cocoașă cu $k \geq 1$, dreapta (aflată strict sub vârful de înălțime 1) intră și iese din cocoașă $\Rightarrow$ exact 2 intersecții, deci $15 \cdot 2 = 30$ soluții.

Pe prima cocoașă $(0, \pi)$: în $x=0$ avem deja soluția comună, iar în interior sinusoida pornește deasupra dreptei ($\sin x \approx x > \frac{x}{100}$) și coboară sub ea la capăt $\Rightarrow$ încă exact 1 soluție.

Total pentru $x > 0$: $30 + 1 = 31$ soluții. Ambele funcții fiind impare, avem prin simetrie 31 de soluții și pentru $x < 0$.

Total: $31 + 31 + 1$ (soluția $x=0$) = **63**.

15. (Limita - forma 1^∞)

Calculați:

$$\lim_{n \rightarrow \infty} \left(\frac{n^2 + 3n + 1}{n^2 + n + 2} \right)^n$$

A) 1

C) e^2 E) ∞ B) e D) e^{-2} F) e^4 **Răspuns corect: C****Rezolvare**

Rescriem baza pentru a evidenția forma $(1 + u)$: Baza = $1 + \frac{2n-1}{n^2+n+2}$.

Suntem în cazul 1^∞ . Exponentul pentru e devine limita produsului:

$$L = \lim_{n \rightarrow \infty} n \cdot \left(\frac{2n-1}{n^2+n+2} \right) = \lim_{n \rightarrow \infty} \frac{2n^2-n}{n^2+n+2} = 2.$$

Prin urmare, limita căutată este e^2 .

INFORMATICĂ

Problema 1

Se consideră următoarele definiții de structuri și inițializarea de mai jos:

```
struct Component {
    int powerLevel;
    bool isActive;
};
struct Device {
    int deviceID;
    char categoryCode;
    Component core;
};

// Inițializare
Device d;
d.deviceID = 404;
d.categoryCode = 'B';
d.core.powerLevel = 0;
d.core.isActive = false;
```

Fiecare operație de mai jos se evaluează **independent**, pornind de fiecare dată de la inițializarea de mai sus. Numărul din față este numărul operației:

```
1 d.categoryCode = d.core.isActive + 64;
2 d.deviceID = d.core;
3 d.core.isActive = d.deviceID;
4 d.core = d.deviceID;
5 d.core.powerLevel = d.categoryCode - 'A';
```

Cerință: Presupunând că fiecare operație numerotată este evaluată independent față de inițializarea de mai sus, care dintre următoarele afirmații este adevărată cu privire la compilare?

- A) Eroare de compilare la **Operația 1** și **Operația 3**
- B) Eroare de compilare la **Operația 2** și **Operația 5**
- C) Eroare de compilare doar la **Operația 4**
- D) Eroare de compilare la **Operația 2** și **Operația 4**
- E) Eroare de compilare la **Operația 1**, **Operația 2** și **Operația 4**
- F) Codul se compilează fără nicio eroare

Partea II: Rezolvare și Barem

Rezolvare

Răspuns corect: D) Eroare de compilare la Operația 2 și Operația 4

Cheia problemei este distincția dintre o *conversie implicită între tipuri aritmetice* (permisă de compilator) și o *atribuire între un tip primitiv și un tip-clasă fără operator de conversie definit* (eroare dură). Analizăm fiecare operație în parte.

Operația 1 — validă. `d.core.isActive` este `false`, adică valoarea întreagă 0. Expresia $0 + 64 = 64$ are tip `int`. Atribuirea unui `int` către un `char` este o conversie implicită permisă (chiar dacă îngustează valoarea): se reține caracterul cu codul 64, adică '@'. Se compilează.

Operația 2 — eroare (Eroarea 1). Se încearcă atribuirea *întregii structuri* `d.core` (tip `Component`) către un `int` (`d.deviceID`). Nu există niciun operator de conversie `Component` $\rightarrow$ `int`, deci compilatorul dă o eroare dură: `cannot convert 'Component' to 'int' in assignment`.

Operația 3 — validă. `d.deviceID` este 404. Atribuirea unui `int` către un `bool` este o conversie implicită permisă: orice valoare nenulă devine `true` (1). Se compilează.

Operația 4 — eroare (Eroarea 2). Reciproca Operației 2. Se încearcă atribuirea unui `int` (`d.deviceID`) către *structura* `d.core` (tip `Component`). Nu există un constructor / operator= care să accepte un `int`, deci compilatorul dă eroare: `no match for 'operator='`.

Operația 5 — validă. `'B' - 'A' = 66 - 65 = 1`, o expresie de tip `int` atribuită unui `int`. Se compilează.

Prin urmare, singurele erori apar la Operațiile 2 și 4 $\Rightarrow$ varianta **D**.

Analiza distractorilor:

- **A)** Presupune că Operațiile 1 și 3 sunt erori — capcana clasică de a crede că o conversie care îngustează (`int` $\rightarrow$ `char`, respectiv `int` $\rightarrow$ `bool`) este interzisă. La atribuire simplă ea este doar o conversie implicită permisă, nu o eroare.
- **B)** Marchează corect Operația 2, dar adaugă greșit Operația 5; studentul confundă aritmetica pe `char` (perfect validă) cu o eroare de tip. Totodată ratează Operația 4.
- **C)** Prinde Operația 4, dar ratează simetria: și Operația 2 (sensul invers, structură $\rightarrow$ `int`) este la fel de invalidă.
- **E)** Adaugă greșit Operația 1 peste răspunsul corect; provine din aceeași confuzie ca la A — că `int` $\rightarrow$ `char` ar fi o eroare.
- **F)** Ignoră complet că o structură nu poate fi atribuită unui/dintr-un tip primitiv fără conversie definită; ar fi corectă doar dacă toate cele 5 operații ar fi conversii aritmetice.

Problema 2

Ce valoare va afișa următorul program scris în C++?

```
#include <iostream>
using namespace std;
int main() {
    int x = 0;
    int y = 5;
    int pasi = 0;
    while (x != y) {
        x = (2 * x + 1) % 17;
        y = (3 * y + 2) % 17;
        pasi++;
    }
    cout << pasi;
    return 0;
}
```

- A) 8
- B) 11
- C) 16
- D) 17
- E) 24
- F) Programul va intra într-o buclă infinită (nu se va opri).

Partea II: Rezolvare și Barem

Rezolvare

Răspuns corect: B) 11

Ideea este să nu simulăm bucla valoare cu valoare, ci să găsim o *formulă închisă* pentru x și y după n pași, în aritmetica modulo 17.

Pasul 1: Formula pentru x . Recurența $x_n = (2x_{n-1} + 1) \bmod 17$ este de forma „înmulțire plus o constantă”. Trucul standard este să găsim numărul fix care, adunat în ambele părți, transformă relația într-o simplă înmulțire repetată. Aici, dacă adunăm 1:

$$x_n + 1 = 2x_{n-1} + 2 = 2(x_{n-1} + 1).$$

Deci la fiecare pas cantitatea $x_n + 1$ se dublează. Pornind de la $x_0 + 1 = 0 + 1 = 1$, după n pași avem $x_n + 1 = 2^n$, adică:

$$x_n \equiv 2^n - 1 \pmod{17}.$$

Pasul 2: Formula pentru y . La fel, $y_n = (3y_{n-1} + 2) \bmod 17$. Adunând tot 1 (numărul fix funcționează la fel):

$$y_n + 1 = 3y_{n-1} + 3 = 3(y_{n-1} + 1).$$

Aici cantitatea $y_n + 1$ se înmulțește cu 3 la fiecare pas. Pornind de la $y_0 + 1 = 5 + 1 = 6$, după n pași avem $y_n + 1 = 6 \cdot 3^n$, adică:

$$y_n \equiv 6 \cdot 3^n - 1 \pmod{17}.$$

Pasul 3: Când devin egale? Bucla se oprește când $x_n = y_n$. Cum ambele formule au „ -1 ”, condiția se simplifică frumos:

$$2^n - 1 \equiv 6 \cdot 3^n - 1 \pmod{17} \iff 2^n \equiv 6 \cdot 3^n \pmod{17}.$$

Nu trebuie nimic sofisticat în continuare: scriem pur și simplu cele două șiruri de puteri modulo 17, unul sub altul, și căutăm prima coloană în care valorile coincid.

n	1	2	3	4	5	6	7	8	9	10	11
$2^n \bmod 17$	2	4	8	16	15	13	9	1	2	4	8
$6 \cdot 3^n \bmod 17$	1	3	9	10	13	5	15	11	16	14	8

Prima valoare a lui n pentru care cele două rânduri sunt egale este **$n = 11$** (ambele dau 8). Atunci:

$$x_{11} \equiv 8 - 1 = 7, \quad y_{11} \equiv 8 - 1 = 7 \pmod{17},$$

deci $x = y = 7$, condiția $x \neq y$ devine falsă și se afișează:

$$\text{pasi} = 11$$

Metodă alternativă (Pasul 3, prin inversul modular)

Dacă stăpânești aritmetica modulară, ecuația de la Pasul 3 poate fi rezolvată direct, fără tabelul cu două rânduri. Pornim de la aceeași condiție:

$$2^n \equiv 6 \cdot 3^n \pmod{17}.$$

Vrem să „scăpăm” de 3^n , adică să împărțim prin el. În aritmetica modulară, a împărți prin 3 înseamnă a înmulți cu *inversul* lui 3. Inversul lui 3 modulo 17 este 6, deoarece

$$3 \cdot 6 = 18 \equiv 1 \pmod{17}.$$

Înmulțim deci ambele părți cu 6^n :

$$2^n \cdot 6^n \equiv 6 \cdot 3^n \cdot 6^n \pmod{17} \iff (2 \cdot 6)^n \equiv 6 \cdot (3 \cdot 6)^n \pmod{17}.$$

Folosind $2 \cdot 6 = 12$ și $3 \cdot 6 = 18 \equiv 1$, partea dreaptă se reduce la $6 \cdot 1^n = 6$, deci totul devine o singură ecuație într-o singură bază:

$$12^n \equiv 6 \pmod{17}.$$

Acum trebuie doar să găsim exponentul n pentru care puterea lui 12 dă 6. Calculăm succesiv:

$$\begin{aligned} 12^1 &\equiv 12, & 12^2 &\equiv 8, & 12^3 &\equiv 11, & 12^4 &\equiv 13, \\ 12^5 &\equiv 3, & 12^6 &\equiv 2, & 12^7 &\equiv 7, & 12^8 &\equiv 16, \\ 12^9 &\equiv 5, & 12^{10} &\equiv 9, & 12^{11} &\equiv \mathbf{6}. \end{aligned}$$

Prima potrivire apare la $\mathbf{n = 11}$, exact ca în metoda cu tabelul. Avantajul acestei căi este că reduce problema la o singură expresie ($12^n \equiv 6$), în loc să comparăm două șiruri în paralel.

Problema 3

Se consideră funcția de mai jos, care primește o matrice a cu n linii și m coloane (unde m este un număr strict par), ambele numerotate începând de la indexul 0. Funcția este proiectată să afișeze elementele parcurgând mai întâi **coloanele de index par** de sus în jos (de la stânga la dreapta). După ce nu mai există coloane la dreapta, trebuie să parcurgă restul coloanelor (cele **impare**) de la dreapta la stânga, fiecare coloană fiind parcursă de jos în sus.

```
void parcurgereMatrice(int a[100][100], int n, int m) {
    if (m % 2 != 0) return; // ne asiguram ca m este par

    // Pasul 1: coloanele pare, de sus in jos
    for (int j = 0; j < m; j += 2) {
        for (int i = 0; i < n; i++) {
            cout << a[i][j] << " ";
        }
    }

    // Pasul 2: restul coloanelor (cele impare)
    // --- COMPLETEAZA AICI (1) ---
    // --- COMPLETEAZA AICI (2) ---
        cout << a[i][j] << " ";
    }
}
}
```

Cerința: Care dintre următoarele perechi de instrucțiuni trebuie plasate în zona marcată (liniile (1) și (2)) pentru ca funcția să realizeze *corect* a doua parte a parcurgerii descrise?

A)

```
(1) for (int j = m - 1; j >= 1; j -= 2) {
(2)     for (int i = n - 1; i >= 0; i--) {
```

B)

```
(1) for (int i = n - 1; i >= 0; i--) {
(2)     for (int j = m - 1; j >= 1; j -= 2) {
```

C)

```
(1) for (int j = 1; j < m; j += 2) {
(2)     for (int i = n - 1; i >= 0; i--) {
```

D)

```
(1) for (int j = m - 2; j >= 0; j -= 2) {
(2)     for (int i = n - 1; i >= 0; i--) {
```

E)

```
(1) for (int j = m; j > 0; j -= 2) {
(2)     for (int i = n; i > 0; i--) {
```

F)

```
(1) for (int j = m - 1; j > 0; j--) {
(2)     for (int i = n - 1; i > 0; i--) {
```

Partea II: Rezolvare și Barem

Rezolvare

Răspuns corect: A)

Pasul 1 — ce coloane rămân după Pasul 1. Deoarece m este par, indecșii coloanelor sunt $0, 1, \dots, m-1$. Coloanele pare $(0, 2, \dots, m-2)$ au fost deja afișate. Rămân coloanele impare $1, 3, \dots, m-1$. Cum m este par, ultimul index $m-1$ este *impar*, deci este exact extremitatea din dreapta de unde trebuie să înceapă parcurgerea de întoarcere.

Pasul 2 — bucla exterioară (coloanele, de la dreapta la stânga). Pornim de la $j = m-1$ și scădem din 2 în 2 ca să nimerim doar coloanele impare $(m-1, m-3, \dots, 1)$, oprindu-ne la $j \geq 1$. Aceasta este linia (1) din varianta A.

Pasul 3 — bucla interioară (liniile, de jos în sus). Pentru fiecare coloană, parcurgem de la $i = n-1$ la $i = 0$ (inclusiv), deci i - cu condiția $i \geq 0$. Aceasta este linia (2) din varianta A. Ordinea buclelor (coloana în exterior, linia în interior) garantează parcurgerea *pe coloane*, nu pe linii.

Verificare pe un exemplu ($n = 3, m = 4$):

1	2	3	4
5	6	7	8
9	10	11	12

Pasul 1 (coloane pare 0, 2 de sus în jos): 1 5 9 3 7 11.

Pasul 2 cu varianta A (coloana 3 jos→sus, apoi coloana 1 jos→sus): 12 8 4 10 6 2. Exact comportamentul cerut.

Analiza distractorilor:

- **B)** Inversează ordinea buclelor: linia în exterior, coloana în interior. Afișează aceleași elemente, dar *pe linii* (row-major), nu pe coloane: 12 10 8 6 4 2. Încalcă cerința de parcurgere pe coloane.
- **C)** Pornește de la $j = 1$ și crește ($j += 2$), deci parcurge coloanele impare de la *stânga la dreapta* (1, 3, ...): 10 6 2 12 8 4. Ignoră cerința de a porni din extremitatea dreaptă.
- **D)** Pornește de la $j = m-2$, care este *par*. Reparcurge din nou coloanele pare $(m-2, m-4, \dots, 0)$ de la coadă la cap: 11 7 3 9 5 1. Index de start greșit.
- **E)** *Out of bounds*. Pornește de la $j = m$ și $i = n$, accesând $a[n][\dots]$ și $a[\dots][m]$, în afara zonei utile a matricei (comportament nedefinit).
- **F)** Folosește j - (pas 1, nu 2), deci atinge și coloanele pare, dublând afișarea. În plus, condiția $i > 0$ ratează elementul de pe linia 0. Rezultat eronat: 12 8 11 7 10 6.

Problema 4

Se consideră următorul program C++:

```
#include <iostream>
using namespace std;

void procesare(int& x) {
    if (x <= 1) {
        return;
    }
    x = x / 2;
    procesare(x);
    x = x * 2;
    cout << x << " ";
}

int main() {
    int n = 50;
    procesare(n);
    return 0;
}
```

Care este lungimea celei mai lungi secvențe de numere nenule (strict pozitive) afișate pe ecran în urma rulării programului?

- A) 6
- B) 0
- C) 5
- D) 25
- E) Eroare de compilare: modificarea parametrului-referință înainte de apelul recursiv este interzisă.
- F) 4

Partea II: Rezolvare și Barem

Rezolvare

Răspuns corect: C) 5

Pasul 1 — observația-cheie: x este referință. Parametrul este declarat `int& x`, iar apelul recursiv se face cu `procesare(x)`. Astfel, toate apelurile de pe stivă lucrează asupra *aceleiași* variabile `n` din `main`; nu există copii separate.

Pasul 2 — coborârea pe stivă (înainte de apel). La fiecare apel se aplică $x = x / 2$ (împărțire întreagă), apoi se recurge. Pornind de la 50:

$$50 \rightarrow 25 \rightarrow 12 \rightarrow 6 \rightarrow 3 \rightarrow 1.$$

Când x ajunge la 1, condiția $x \leq 1$ oprește recursivitatea fără a mai face un apel nou și fără a afișa nimic (cazul de bază nu conține `cout`).

Pasul 3 — urcarea pe stivă (după apel). La revenirea din fiecare apel se execută $x = x * 2$, urmat de afișare. Deoarece x este aceeași variabilă, valoarea de pornire la urcare este 1:

$$1 \cdot 2 = 2, \quad 2 \cdot 2 = 4, \quad 4 \cdot 2 = 8, \quad 8 \cdot 2 = 16, \quad 16 \cdot 2 = 32.$$

Pasul 4 — ieșirea efectivă. Programul afișează:

$$2 \quad 4 \quad 8 \quad 16 \quad 32$$

o secvență de 5 numere, toate strict pozitive. Prin urmare lungimea cerută este 5.

Analiza distractorilor:

- **A) 6** — apare la cei care numără și cazul de bază (valoarea 1) ca pe o afișare. Este greșit: `cout` este *după* apelul recursiv, iar la $x = 1$ funcția face `return` fără să afișeze.
- **B) 0** — capcana „referința distruge numărul”: elevii care anticipează un truc cred că efectul de referință anulează cumva valorile. Condiția $x \leq 1$ oprește exact înainte ca valoarea să devină 0, deci afișările au loc normal.
- **D) 25** — răspunsul „leneș” $N/2 = 50/2$, dat de cei care presupun că numărul de afișări este jumătate din valoarea inițială, ignorând că pasul recursiv este o *înjumătățire*, nu o scădere cu 1.
- **E)** — distractor sintactic fals. Modificarea unui parametru transmis prin referință înainte de apelul recursiv este perfect legală în C++; codul compilează fără erori.
- **F) 4** — apare la cei care „pierd” o împărțire din cauza numerelor impare (de exemplu sar peste $25 \rightarrow 12$ sau $3 \rightarrow 1$), numărând doar 4 niveluri în loc de 5.

Problema 5

Se consideră un **arbore generic complet** cu 2500 de noduri, în care un nod poate avea *oricâți* fii. Nodurile sunt parcurse în lățime (pe niveluri, de la stânga la dreapta) și numerotate cu numere naturale consecutive, începând de la rădăcină, care este nodul 1 și se află pe nivelul 1. Copilul stâng al rădăcinii este nodul 2, copilul drept al rădăcinii este nodul 3 și tot așa.

Regula de ramificare este următoarea: orice nod etichetat cu valoarea k are exact $k + 1$ descendenți direcți (fii). Astfel, nodul 1 are 2 fii, nodul 2 are 3 fii, nodul 3 are 4 fii și așa mai departe, fiii fiind atribuiți în ordinea crescătoare a etichetelor, până la epuizarea celor 2500 de noduri.

Câte niveluri are, în total, acest arbore?

- A) 5
- B) 6
- C) 4
- D) 8
- E) 7
- F) 9

Partea II: Rezolvare și Barem

Rezolvare

Răspuns corect: B) 6

Notăm cu M_L *indicele maxim* al unui nod aflat pe nivelul L — adică numărul total de noduri din arbore până la nivelul L inclusiv. Nodul 2500 se află pe primul nivel L pentru care $M_L \geq 2500$. Cheia este următoarea observație: toți fiii nodurilor de pe nivelul L formează nivelul $L + 1$. Dacă nivelul L ocupă etichetele $a, a + 1, \dots, b$ (deci $a = M_{L-1} + 1$ și $b = M_L$), numărul de noduri noi de pe nivelul $L + 1$ este

$$\sum_{k=a}^b (k+1) = \sum_{j=a+1}^{b+1} j.$$

Nivelul 1. Doar rădăcina: nodul 1. Deci $M_1 = 1$.

Nivelul 2. Nodul 1 are 2 fii (nodurile 2, 3). $M_2 = 3$.

Nivelul 3. Fiii nodurilor 2 și 3: $3 + 4 = 7$ noduri noi (nodurile 4–10). $M_3 = 10$.

Nivelul 4. Fiii nodurilor 4–10: $\sum_{k=4}^{10} (k+1) = \sum_{j=5}^{11} j = 56$ noduri noi (nodurile 11–66). $M_4 = 66$.

Nivelul 5. Fiii nodurilor 11–66:

$$\sum_{k=11}^{66} (k+1) = \sum_{j=12}^{67} j = \frac{67 \cdot 68}{2} - \frac{11 \cdot 12}{2} = 2278 - 66 = 2212 \text{ noduri noi.}$$

Acestea sunt nodurile 67–2278, deci $M_5 = 66 + 2212 = 2278$. (Observație: cei doi termeni 66 se anulează, așa că $M_5 = \frac{67 \cdot 68}{2} = 2278$, un rezultat „curat” prin suma lui Gauss.)

Nivelul 6. Avem $M_5 = 2278 < 2500$, deci nodul 2500 *nu* încapă pe nivelul 5. Mai rămân $2500 - 2278 = 222$ de noduri, care sunt fiii primelor noduri de pe nivelul 5 (nodurile 67, 68, 69, 70, ...). Aceste 222 de noduri „se varsă” pe nivelul 6 (nodurile 2279–2500), formând un ultim nivel incomplet, dar perfect valid pentru un arbore generic complet (umplut de la stânga la dreapta).

Concret, fiii se distribuie astfel pe nivelul 6: nodul 67 dă 68 fii (2279–2346), nodul 68 dă 69 fii (2347–2415), nodul 69 dă 70 fii (2416–2485), iar nodul 70 (71 fii) îi furnizează pe 2486–2556 — dar ne oprim la 2500. Prin urmare, nodul 2500 este al 15-lea fiu al nodului 70.

Deoarece nodul 2500 apare pentru prima dată pe nivelul 6, arborele are în total $\boxed{6}$ niveluri.

Analiza distractorilor:

- **A) 5:** candidatul calculează corect $M_5 = 2278$ (numărul „rotund” dat de suma Gauss $\frac{67 \cdot 68}{2}$), dar uită să observe că $2278 < 2500$ și nu adaugă nivelul pe care „se varsă” cele 222 de noduri rămase. Eroare clasică de tip off-by-one (oprire cu un nivel mai devreme).
- **C) 4:** candidatul *supraestimează* dimensiunea unui nivel timpuriu (de exemplu greșește suma de pe nivelul 4 sau 5, crezând-o mult mai mare), așa că în calculul lui cele 2500 de noduri par să se epuizeze deja pe nivelul 4 — se oprește cu un nivel prea devreme.
- **D) 8:** eroarea-oglindă a lui C. Candidatul *subestimează* câți fii aduc nodurile cu etichetă mare (uită că numărul de fii crește odată cu eticheta: nodul 66 are deja 67 fii), deci arborele pare să crească lent. Astfel îi atribuie artificial mai multe niveluri pentru a ajunge la 2500, supraestimând adâncimea (≈ 8).

- **E) 7:** capcana de frontieră. Candidatul tratează nodul 2500 ca pe un nod *intern* și adaugă încă un nivel (al 7-lea) pentru cei 2501 fii pe care regula i-ar atribui — deși nodul 2500 este ultimul din arbore, deci o *frunză* fără niciun fiu prezent. Confundă fiii „teoretici” cu fiii reali.
- **F) 9:** candidatul face o eroare aritmetică în suma dominantă de pe nivelul 5 — de exemplu împarte la 2 o dată în plus, aplicând formula lui Gauss de două ori — și obține o contribuție mult mai mică (sute, în loc de 2212). Astfel arborele pare să crească mult mai lent, iar candidatul prelungește recurența până pe la nivelul 9 înainte ca totalul cumulat să depășească 2500.

Problema 6

Considerăm grafuri neorientate **simple** (fără bucle și fără muchii multiple) cu 5 noduri **etichetate** (numerotate de la 1 la 5).

Reamintim:

- un graf este **eulerian** dacă este conex și toate nodurile au grad par (echivalent: admite un circuit eulerian);
- un graf este **hamiltonian** dacă conține un ciclu care trece prin fiecare nod exact o dată.

Câte grafuri cu 5 noduri etichetate sunt simultan euleriene și hamiltoniene?

- A) 12
- B) 13
- C) 24
- D) 1
- E) 0
- F) 120

Partea II: Rezolvare și Barem

Rezolvare

Răspuns corect: B) 13

Pasul 1: ce grade sunt posibile. Într-un graf simplu cu 5 noduri, gradul fiecărui nod este între 0 și 4. Condiția de *eulerian* cere ca toate gradele să fie *pare*, deci fiecare grad este 0, 2 sau 4. Condiția de conexitate elimină gradul 0 (un nod izolat ar deconecta graful). Prin urmare fiecare nod are gradul 2 sau 4.

Pasul 2: clasificarea după numărul de noduri de grad 4. Fie k numărul de noduri de grad 4. Suma gradelor este $4k + 2(5 - k) = 2k + 10$, deci numărul de muchii este $m = k + 5$. Analizăm fiecare valoare $k \in \{0, 1, 2, 3, 4, 5\}$:

$k = 0$ — secvența (2, 2, 2, 2, 2). Un graf conex în care toate nodurile au grad 2 este un *singur ciclu*; pe 5 noduri singura posibilitate este ciclul C_5 . Este chiar el propriul ciclu hamiltonian, deci *eulerian și hamiltonian*. Numărul de cicluri C_5 pe noduri etichetate este

$$\frac{(5-1)!}{2} = \frac{4!}{2} = \frac{24}{2} = 12.$$

(Împărțim la 2 deoarece graful e neorientat: parcurgerea în cele două sensuri dă același ciclu.)
 $\Rightarrow$ **12 grafuri.**

$k = 1$ — secvența (4, 2, 2, 2, 2). Nodul de grad 4 este conectat la toate celelalte; fiecare dintre celelalte patru noduri mai are nevoie de o muchie, formând un cuplaj perfect între ele. Astfel de grafuri *există* (sunt 15), dar *niciunul nu este hamiltonian*: fiecare nod de grad 2 are ca vecini doar nodul central și perechea sa, deci într-un ciclu hamiltonian ar fi obligat să folosească ambele muchii; atunci nodul central ar trebui să fie vecin în ciclu cu toate cele patru noduri, imposibil (un nod are exact 2 vecini în ciclu). $\Rightarrow$ **0.**

$k = 2$ — secvența (4, 4, 2, 2, 2). Cele două noduri de grad 4 sunt conectate la toate; cele trei noduri de grad 2 se conectează doar la acestea două (nu au muchii între ele). Astfel de grafuri *există* (sunt 10), dar *niciunul nu este hamiltonian*: cele trei noduri de grad 2 ar trebui, fiecare, să fie adiacente în ciclu ambelor noduri centrale ($3 \times 2 = 6$ “sloturi”), însă cele două noduri centrale oferă doar $2 + 2 = 4$ poziții de vecin în ciclu — principiul cutiei interzice acest lucru. $\Rightarrow$ **0.**

$k = 3$ — secvența (4, 4, 4, 2, 2). Aceasta *nu este realizabilă*. Un nod de grad 4 (pe doar 5 noduri) este conectat la *toate* celelalte. Dacă trei noduri au grad 4, atunci fiecare dintre celelalte două este conectat la toate aceste trei noduri, deci ar avea grad ≥ 3 , în contradicție cu gradul 2. Nu există niciun graf cu această secvență. $\Rightarrow$ **0.**

$k = 4$ — secvența (4, 4, 4, 4, 2). La fel, *nerealizabilă*: cele patru noduri de grad 4 sunt conectate la toate, deci al cincilea nod este conectat la toate patru, având grad 4, nu 2. $\Rightarrow$ **0.**

$k = 5$ — secvența (4, 4, 4, 4, 4). Este graful complet K_5 . Toate gradele sunt 4 (pare) și graful e conex, deci eulerian; orice graf complet cu $n \geq 3$ noduri este hamiltonian. Există un singur K_5 pe noduri etichetate. $\Rightarrow$ **1 graf.**

Pasul 3: total. Singurele configurații care îndeplinesc simultan ambele condiții sunt ciclurile C_5 și graful complet K_5 :

$$12 + 1 = \boxed{13}.$$

Analiza distractorilor:

- **A)** 12 — numără corect familia ciclurilor C_5 , dar uită complet graful complet K_5 , care este și el eulerian și hamiltonian.
- **C)** 24 — folosește formula $(n - 1)! = 4! = 24$ pentru ciclurile hamiltoniene, uitând să împartă la 2 (graf neorientat) — și, în plus, ignoră K_5 .
- **D)** 1 — se gândește doar la cazul “evident” K_5 și ratează cele 12 cicluri C_5 .
- **E)** 0 — presupune greșit că cele două proprietăți se exclud reciproc (confuzie între circuit eulerian și ciclu hamiltonian); de fapt C_5 și K_5 le satisfac pe ambele.
- **F)** 120 — calculează $5! = 120$, numărul de permutări ale etichetelor, fără legătură cu structura cerută a grafului.

Problema 7

Se consideră următorul program în C/C++:

```
#include <iostream>
#include <string.h>
using namespace std;
int main() {
    char s[] = "automatica_calculatoare_cti";
    char ref[] = "calculatoare";
    char rez[50] = "";
    char *t = strtok(s, "_");
    while (t != NULL) {
        int comp = strcmp(t, ref);
        if (comp < 0) {
            strcat(rez, t + 4);
        } else if (comp > 0) {
            char m[10];
            strcpy(m, t);
            m[strlen(t) - 1] = '\\0';
            strcat(rez, m);
        } else {
            rez[2] = '\\0';
        }
        t = strtok(NULL, "_");
    }
    cout << rez;
    return 0;
}
```

Cerința: Ce se va afișa pe ecran în urma rulării programului?

- A) maticact
- B) mact
- C) omct
- D) ma
- E) automatica_calculatoare_cti
- F) Eroare de rulare (Segmentation Fault)

Partea II: Rezolvare și Barem

Rezolvare

Răspuns corect: B) mact

Inițializare. `strtok` împarte `s` în token-uri folosind separatorul `_`, înlocuind fiecare separator cu `'\0'`. Token-urile obținute, în ordine, sunt: "automatica", "calculatoare", "cti". La început `rez` este șirul vid `""`.

Pasul 1: `t = "automatica".strcmp("automatica", "calculatoare");` prima literă diferă, `'a'` (cod 97) < `'c'` (cod 99), deci rezultatul este *strict negativ* $\Rightarrow$ ramura `comp < 0`. Se execută `strcat(rez, t + 4)`. Pointerul `t + 4` sare peste primele 4 caractere (a,u,t,o) și indică spre "matica".

`rez = "matica".`

Pasul 2: `t = "calculatoare".` Șirurile sunt identice, deci `strcmp` returnează 0 $\Rightarrow$ ramura `else: rez[2] = '\0'`. Cum `rez` era "matica", caracterul de pe poziția 2 (`'t'`) devine terminatorul de șir, deci `rez` se trunchiază:

`rez = "ma".`

Pasul 3: `t = "cti".strcmp("cti", "calculatoare");` *capcana de lungime*. Compararea se face caracter cu caracter, nu după lungime. Prima literă coincide (`'c' == 'c'`), dar a doua diferă: `'t'` (cod 116) > `'a'` (cod 97). Prin urmare "cti" este *mai mare* decât "calculatoare", iar `strcmp` returnează o valoare *pozitivă* $\Rightarrow$ ramura `else if`. Se copiază "cti" în `m`; apoi `m[strlen(t) - 1] = m[2] = '\0'` înlocuiește `'i'` cu terminatorul, deci `m = "ct"`. În final `strcat(rez, m)` adaugă "ct":

`rez = "ma" + "ct" = "mact".`

Final. `strtok(NULL, "_")` returnează NULL, bucla se încheie și se afișează mact.

Analiza distractorilor:

- **A) maticact** — candidatul ratează complet instrucțiunea de trunchiere `rez[2] = '\0'` de la token-ul egal; `rez` rămâne "matica", iar la pasul 3 se adaugă "ct", dând "maticact".
- **C) omct** — greșală de offset la `t + 4`: candidatul numără pornind de la poziția greșită (consideră că `t+4` indică spre `'o'`), obținând "omatica"; după trunchiere rezultă "om", apoi + "ct" dă "omct".
- **D) ma** — candidatul cade în capcana de lungime la `strcmp` pentru "cti", crezând că un șir mai scurt este "mai mic". Ar lua atunci ramura `comp < 0` și ar aplica `t + 4` pe un token de lungime 3 (depășind sfârșitul șirului), presupunând că nu se adaugă nimic; `rez` rămâne "ma".
- **E) automatica_calculatoare_cti** — candidatul crede că programul reconstruiește sau afișează șirul inițial, ignorând că se afișează `rez` (construit din fragmente) și că `strtok` a înlocuit deja separatorii `_` cu `'\0'` în `s`.
- **F) Segmentation Fault** — candidatul presupune un crash, fie confundând `char s[]` (tablou modificabil pe stivă) cu `char *s` (literal read-only) pe care `strtok` ar încerca să-l modifice, fie crezând că accesul `t + 4` pe un token scurt produce automat eroare. Pe execuția corectă nu se atinge memorie invalidă, deci programul rulează fără excepție.

Problema 8

Se consideră următorul program C++:

```
#include <iostream>
using namespace std;

int apeluri = 0;

int fib(int n) {
    int memo[30] = {0};
    if (n < 2)
        return n;
    if (memo[n] != 0)
        return memo[n];
    apeluri++;
    memo[n] = fib(n - 1) + fib(n - 2);
    return memo[n];
}

int main() {
    fib(12);
    cout << apeluri << endl;
    return 0;
}
```

Ce valoare va afișa programul?

- A) 11
- B) 144
- C) 232
- D) 233
- E) 287
- F) 465

Partea II: Rezolvare și Barem

Rezolvare

Răspuns corect: C) 232

Pasul 1: tabelul memo nu memoizează nimic.

Vectorul `memo` este declarat *în interiorul* funcției `fib`, deci este o variabilă locală: fiecare apel primește propriul vector, inițializat cu zero prin `= {0}`, iar la ieșirea din funcție acesta dispare. Valorile scrise în `memo[n]` nu supraviețuiesc apelului care le-a scris, așa că testul `memo[n] != 0` este fals de fiecare dată. Recursia se comportă exact ca varianta naivă, cu arbore de apeluri exponențial.

Pasul 2: structura arborelui de apeluri.

Apelul `fib(n)` generează un arbore binar în care:

- **frunzele** sunt apelurile cu $n < 2$, care returnează imediat;
- **nodurile interne** sunt apelurile cu $n \geq 2$, care generează la rândul lor doi fii.

Fie $L(n)$ numărul de frunze ale arborelui lui `fib(n)`. Avem $L(0) = L(1) = 1$ și $L(n) = L(n-1) + L(n-2)$ pentru $n \geq 2$ — aceeași recurență ca Fibonacci, cu aceleași valori inițiale ca $F(1) = F(2) = 1$. Prin urmare

$$L(n) = F(n+1).$$

Un arbore binar strict (fiecare nod are 0 sau 2 fii) cu L frunze are exact $L - 1$ noduri interne. Deci numărul de noduri interne este

$$I(n) = F(n+1) - 1,$$

iar numărul total de apeluri este $L(n) + I(n) = 2F(n+1) - 1$.

Pasul 3: ce numără de fapt contorul.

Instrucțiunea `apeluri++` se află *după* testul cazului de bază. Un apel cu $n < 2$ execută `return n`; și iese din funcție fără să treacă vreodată pe linia contorului. Așadar contorul nu numără toate apelurile, ci doar nodurile interne:

$$\text{apeluri} = I(12) = F(13) - 1.$$

Pasul 4: calculul.

Șirul lui Fibonacci cu $F(1) = F(2) = 1$:

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233$$

deci $F(13) = 233$ și

$$\text{apeluri} = 233 - 1 = \boxed{232}.$$

(Verificare de consistență: arborele are $L(12) = F(13) = 233$ frunze și 232 noduri interne, în total 465 apeluri, dintre care doar 232 ajung la contor.)

Problema 9

Andrei are în portofel exact 3 bancnote de 1 leu, 6 bancnote de 2 lei, 4 bancnote de 5 lei și 5 bancnote de 10 lei (în total 85 de lei). El vrea să plătească suma de 20 de lei folosind numai bancnotele pe care le are, fără a primi rest.

Două plăți se consideră diferite dacă diferă prin numărul de bancnote folosite din cel puțin un tip (bancnotele de aceeași valoare sunt indistinguibile, iar ordinea în care sunt înmânate nu contează).

În câte moduri poate Andrei să plătească exact 20 de lei?

- A) 9
- B) 11
- C) 12
- D) 15
- E) 35
- F) 40

Partea II: Rezolvare și Barem

Rezolvare

Răspuns corect: C) 12

Pasul 1: Modelarea problemei. Fie a, b, c, d numărul de bancnote folosite de 1, 2, 5, respectiv 10 lei. Se cere numărul soluțiilor în numere întregi ale sistemului

$$a + 2b + 5c + 10d = 20, \quad 0 \leq a \leq 3, \quad 0 \leq b \leq 6, \quad 0 \leq c \leq 4, \quad 0 \leq d \leq 5.$$

Echivalent, răspunsul este coeficientul lui x^{20} în funcția generatoare cu factori *finiți*

$$G(x) = \underbrace{(1 + x + x^2 + x^3)}_{1 \text{ leu}} \cdot \underbrace{(1 + x^2 + \dots + x^{12})}_{2 \text{ lei}} \cdot \underbrace{(1 + x^5 + x^{10} + x^{15} + x^{20})}_{5 \text{ lei}} \cdot \underbrace{(1 + x^{10} + \dots + x^{50})}_{10 \text{ lei}}.$$

Faptul că fiecare factor este un polinom, nu o serie geometrică infinită, este esențial: plafoanele de stoc elimină soluții.

Pasul 2: Restrângerea cazurilor. Din $10d \leq 20$ rezultă $d \in \{0, 1, 2\}$ (plafonul $d \leq 5$ nu este activ). Analog, din $5c \leq 20$ rezultă $c \leq 4$, deci nici plafonul bancnotelor de 5 lei nu este activ. Constrângerile care taie efectiv sunt $a \leq 3$ și $b \leq 6$.

Pasul 3: O observație de paritate. Modulo 2, relația devine $a \equiv 20 - 2b - 5c - 10d \equiv c \pmod{2}$. Așadar a și c au aceeași paritate. Cum $a \leq 3$, avem $a \in \{0, 2\}$ când c este par și $a \in \{1, 3\}$ când c este impar. Pentru fiecare pereche (a, c, d) admisibilă, valoarea lui b este unic determinată:

$$b = \frac{20 - a - 5c - 10d}{2},$$

deci mai rămâne doar verificarea condițiilor $b \geq 0$ și $b \leq 6$.

Pasul 4: Numărarea după d .

Cazul $d = 2$: rămâne $a + 2b + 5c = 0$, deci $a = b = c = 0$.

1 soluție

Cazul $d = 1$: rămâne $a + 2b + 5c = 10$, cu $c \leq 2$.

- $c = 0$: $a \in \{0, 2\}$, de unde $b = 5$, respectiv $b = 4$; ambele au $b \leq 6$. (2)
- $c = 1$: $a \in \{1, 3\}$, de unde $b = 2$, respectiv $b = 1$. (2)
- $c = 2$: $a = 0 \Rightarrow b = 0$; $a = 2$ ar da $2b = -2$, imposibil. (1)

5 soluții

Cazul $d = 0$: rămâne $a + 2b + 5c = 20$, cu $c \leq 4$.

- $c = 0$: $b = (20 - a)/2 \in \{10, 9\}$ pentru $a \in \{0, 2\}$ — ambele depășesc plafonul $b \leq 6$. (0)
- $c = 1$: $a \in \{1, 3\}$ dă $b = 7$, respectiv $b = 6$; doar $a = 3$ este acceptat. (1)
- $c = 2$: $a \in \{0, 2\}$ dă $b = 5$, respectiv $b = 4$. (2)
- $c = 3$: $a \in \{1, 3\}$ dă $b = 2$, respectiv $b = 1$. (2)
- $c = 4$: $a = 0 \Rightarrow b = 0$; $a = 2$ ar cere $2b = -2$, imposibil. (1)

6 soluții**Pasul 5: Totalul.**

$$1 + 5 + 6 = \boxed{12}$$

Verificare (lista completă a plăților (a, b, c, d)):

(0, 0, 0, 2)	(0, 0, 2, 1)	(0, 0, 4, 0)	(0, 5, 0, 1)
(0, 5, 2, 0)	(1, 2, 1, 1)	(1, 2, 3, 0)	(2, 4, 0, 1)
(2, 4, 2, 0)	(3, 1, 1, 1)	(3, 1, 3, 0)	(3, 6, 1, 0)

Problema 10

Analizați următorul fragment de cod:

```
for (int i = 1; i <= n; i *= 2)
    for (int j = 1; j <= i; j *= 2)
        cout << " * ";
```

Care este complexitatea de timp a fragmentului, în funcție de n ?

- A) $O(n)$
- B) $O(\log n)$
- C) $O(\log^2 n)$
- D) $O(n \log n)$
- E) $O(\sqrt{n})$
- F) $O(1)$

Partea II: Rezolvare și Barem

Rezolvare

Răspuns corect: C) $O(\log^2 n)$

Pasul 1: bucla exterioară. Variabila i pornește de la 1 și se *dublează* la fiecare pas ($i *= 2$), luând valorile $1, 2, 4, \dots, 2^k$, unde $2^k \leq n$. Prin urmare bucla exterioară se execută de $k + 1$ ori, cu $k = \lfloor \log_2 n \rfloor$, adică de ordinul $\Theta(\log n)$ ori.

Pasul 2: bucla interioară depinde de i . Capcana este că bucla interioară *nu* are un număr fix de pași: limita ei superioară este chiar i . Pentru o valoare fixată $i = 2^t$, variabila j ia valorile $1, 2, 4, \dots, 2^t$, deci bucla interioară se execută $t + 1$ ori.

Pasul 3: însumarea. Numărul total de caractere $*$ afișate (deci numărul total de pași) este suma, peste toate iterațiile exterioare $t = 0, 1, \dots, k$, a numărului de pași interiori:

$$\sum_{t=0}^k (t+1) = 1 + 2 + \dots + (k+1) = \frac{(k+1)(k+2)}{2}.$$

Pasul 4: concluzia. Cu $k = \lfloor \log_2 n \rfloor$, suma este de ordinul $\Theta(k^2) = \Theta(\log^2 n)$. Așadar complexitatea de timp este

$$\boxed{O(\log^2 n)},$$

adică varianta **C**.

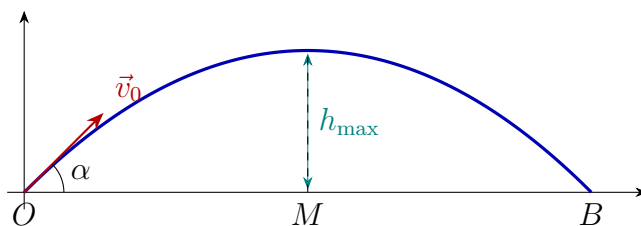
Analiza distractorilor:

- **A)** $O(n)$ — apare la cei care confundă incrementarea *multiplicativă* ($i *= 2$) cu una *aditivă* ($i++$); cu $i++$ bucla exterioară ar rula de n ori, dar aici i crește exponențial, deci se execută doar logaritm de multe ori.
- **B)** $O(\log n)$ — numără corect bucla exterioară, dar tratează bucla interioară ca având cost *constant*, ignorând că numărul ei de pași ($t + 1$) crește odată cu i . Se pierde astfel al doilea factor logaritm.
- **D)** $O(n \log n)$ — combinație a două greșeli: presupune bucla exterioară liniară ($O(n)$, ca la $i++$) și cea interioară logaritmă. Este complexitatea tipică a unui **for** liniar cu un **for** interior care se dublează — dar nu a codului de față.
- **E)** $O(\sqrt{n})$ — nu are nicio bază în structura buclelor; provine din asocierea mecanică a dublării cu radicalul, fără a efectua însumarea reală a pașilor.
- **F)** $O(1)$ — presupune greșit că numărul de stele este mărginit de o constantă; de fapt el crește nelimitat (pătratic în $\log n$) odată cu n .

FIZICĂ

Mecanică

1. (*Aruncare oblică*) Un corp este aruncat după o direcție care face unghiul α cu orizontala. Să se afle acest unghi, știind că distanța parcursă de corp pe orizontală este de patru ori mai mare decât înălțimea maximă atinsă de el pe traiectorie. Se neglijează interacțiunea cu aerul.

A) 45° C) $\arctg 2$ E) 40° B) $\arctg \frac{4}{5}$ D) 30° F) $\arctg \frac{1}{2}$

Rezolvare

Distanța parcursă pe orizontală (bătaia) și înălțimea maximă sunt date de ecuațiile:

$$D = \frac{v_0^2 \sin(2\alpha)}{g} = \frac{2v_0^2 \sin \alpha \cos \alpha}{g}$$

$$h_{\max} = \frac{v_0^2 \sin^2 \alpha}{2g}$$

Ni se dă condiția $D = 4h_{\max}$. Înlocuind formulele:

$$\frac{2v_0^2 \sin \alpha \cos \alpha}{g} = 4 \left(\frac{v_0^2 \sin^2 \alpha}{2g} \right)$$

Simplificând termenii comuni (v_0^2 și g) obținem:

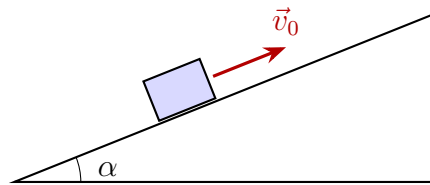
$$2 \sin \alpha \cos \alpha = 2 \sin^2 \alpha$$

Împărțind prin $2 \sin \alpha \cos \alpha$ (știind că unghiul $\alpha \neq 0$), rezultă:

$$\operatorname{tg} \alpha = 1 \implies \alpha = 45^\circ$$

Răspuns corect: A

2. (*Plan înclinat cu frecare*) Un corp este lansat cu viteza inițială $v_0 = 35 \text{ m/s}$ pe un plan înclinat care formează cu orizontala un unghi $\alpha = 30^\circ$. Știind că raportul dintre timpul de coborâre și cel de urcare este $\sqrt{7}$, să se calculeze viteza finală a corpului cu care trece prin punctul de lansare.



A) $5\sqrt{3}$ m/s

C) $5\sqrt{7}$ m/s

E) 17,5 m/s

B) 35 m/s

D) 12,24 m/s

F) $7\sqrt{7}$ m/s

Rezolvare

Accelerațiile pe planul înclinat sunt diferite la urcare față de coborâre:

- Urcare: $a_u = g(\sin \alpha + \mu \cos \alpha)$
- Coborâre: $a_c = g(\sin \alpha - \mu \cos \alpha)$

Timpul de urcare până la oprire este $t_u = \frac{v_0}{a_u}$. Distanța parcursă până la oprire este $d = \frac{v_0^2}{2a_u}$.

La coborâre, pornește din repaus, deci $d = \frac{a_c t_c^2}{2}$. Egalând distanțele, obținem timpul de coborâre:

$$t_c = \sqrt{\frac{2d}{a_c}} = \sqrt{\frac{v_0^2}{a_u a_c}} = \frac{v_0}{\sqrt{a_u a_c}}$$

Raportul timpurilor dat în problemă este:

$$\frac{t_c}{t_u} = \frac{\frac{v_0}{\sqrt{a_u a_c}}}{\frac{v_0}{a_u}} = \sqrt{\frac{a_u}{a_c}} = \sqrt{7}$$

Viteza finală la baza planului se obține din legea lui Galilei pentru coborâre ($v_f^2 = 2a_c d$):

$$v_f = \sqrt{2a_c \frac{v_0^2}{2a_u}} = v_0 \sqrt{\frac{a_c}{a_u}}$$

Observăm că termenul de sub radical este exact inversul raportului timpurilor. Așadar:

$$v_f = v_0 \frac{t_u}{t_c} = \frac{35}{\sqrt{7}} = \frac{35\sqrt{7}}{7} = 5\sqrt{7} \text{ m/s}$$

Răspuns corect: C

3. (Ciocnire plastică) Un corp de masă $m_1 = 2$ kg, având viteza $v_1 = 6$ m/s, lovește plastic (ciocnire perfect inelastică) un corp de masă $m_2 = 4$ kg aflat în repaus. Energia cinetică pierdută în ciocnire este:

A) 12 J

C) 24 J

E) 6 J

B) 36 J

D) 0 J

F) 18 J

Rezolvare

Aplicăm legea conservării impulsului pentru a afla viteza comună a sistemului după cioc-

nire:

$$m_1 v_1 = (m_1 + m_2) v' \implies v' = \frac{m_1 v_1}{m_1 + m_2} = \frac{2 \cdot 6}{2 + 4} = 2 \text{ m/s}$$

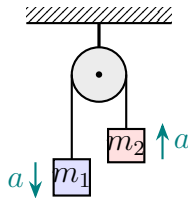
Energia cinetică pierdută se calculează ca diferența dintre energia cinetică inițială și cea finală:

$$\Delta E_c = E_{c,i} - E_{c,f} = \frac{m_1 v_1^2}{2} - \frac{(m_1 + m_2)(v')^2}{2}$$

$$\Delta E_c = \frac{2 \cdot 36}{2} - \frac{6 \cdot 4}{2} = 36 - 12 = 24 \text{ J}$$

Răspuns corect: C

4. (*Dinamica corpurilor legate*) Două corpuri de mase $m_1 = 3 \text{ kg}$ și $m_2 = 2 \text{ kg}$ sunt legate printr-un fir inextensibil și de masă neglijabilă, trecut peste un scripete ideal (fără frecare, fără masă). Sistemul este lăsat liber. Se dă $g = 10 \text{ m/s}^2$. Tensiunea din fir este:



A) 10 N

C) 12 N

E) 30 N

B) 50 N

D) 24 N

F) 30 kg

Rezolvare

Sistemul este o mașină Atwood. Scriem ecuațiile de mișcare pentru fiecare corp, alegând sensul de mișcare în direcția corpului mai greu (m_1 coboară, m_2 urcă):

$$m_1 g - T = m_1 a$$

$$T - m_2 g = m_2 a$$

Adunând ecuațiile obținem accelerația:

$$a = g \frac{m_1 - m_2}{m_1 + m_2} = 10 \frac{3 - 2}{3 + 2} = 10 \cdot \frac{1}{5} = 2 \text{ m/s}^2$$

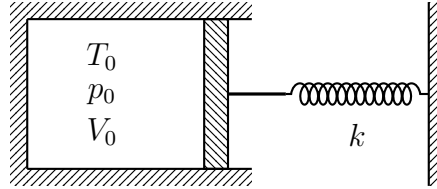
Din a doua ecuație aflăm tensiunea:

$$T = m_2(g + a) = 2 \cdot (10 + 2) = 24 \text{ N}$$

Răspuns corect: D

Termodinamică

5. (*Gaz + piston cuplat cu arc*) Într-un cilindru cu piston se găsește un volum de gaz $V_0 = 2$ L la $p_0 = 10^5$ Pa și $t_0 = 27^\circ\text{C}$. Pistonul poate fi comprimat de un arc cu coeficientul elastic $k = 1000$ N/m, aria pistonului fiind $S = 1$ dm². Se încălzește gazul cu $\Delta t = 50$ K. Să se calculeze comprimarea Δl a arcului în urma încălzirii. Se consideră $\sqrt{5664} \approx 75,25$.



- A) 10 cm C) 0,2 m E) 2,7 cm
 B) 0,2 cm D) 0,27 m F) 0,27 cm

Rezolvare

Notăm starea inițială cu $p_0, V_0, T_0 = 300$ K și starea finală cu $p_1, V_1, T_1 = 350$ K.

Dilatarea gazului comprimă arcul cu distanța Δl . Noul volum este $V_1 = V_0 + S\Delta l$.

Noua presiune trebuie să învingă atât presiunea inițială, cât și forța elastică a arcului raportată la suprafață:

$$p_1 = p_0 + \frac{k\Delta l}{S}$$

Aplicăm ecuația de stare pentru masa constantă de gaz:

$$\frac{p_0 V_0}{T_0} = \frac{p_1 V_1}{T_1} \implies \frac{p_0 V_0}{T_0} = \frac{(p_0 + \frac{k\Delta l}{S})(V_0 + S\Delta l)}{T_1}$$

Convertim toate datele în Sistemul Internațional:

- $V_0 = 2 \cdot 10^{-3} \text{ m}^3$
- $S = 10^{-2} \text{ m}^2$
- $\frac{p_0 V_0}{T_0} = \frac{10^5 \cdot 2 \cdot 10^{-3}}{300} = \frac{2}{3}$

Înlocuim în ecuația de stare:

$$\frac{2}{3} = \frac{(10^5 + \frac{1000\Delta l}{10^{-2}})(2 \cdot 10^{-3} + 10^{-2}\Delta l)}{350}$$

$$\frac{2}{3} = \frac{10^5(1 + \Delta l) \cdot 10^{-2}(0,2 + \Delta l)}{350}$$

$$\frac{2}{3} = \frac{1000(1 + \Delta l)(0,2 + \Delta l)}{350} = \frac{20}{7}(1 + \Delta l)(0,2 + \Delta l)$$

$$14 = 60(0,2 + 1,2\Delta l + \Delta l^2) \implies 60\Delta l^2 + 72\Delta l - 2 = 0$$

Rezolvăm ecuația de gradul al doilea:

$$\Delta = 72^2 - 4 \cdot 60 \cdot (-2) = 5184 + 480 = 5664$$

$$\Delta l = \frac{-72 + \sqrt{5664}}{120} \approx \frac{-72 + 75,25}{120} = \frac{3,25}{120} \approx 0,027 \text{ m} = 2,7 \text{ cm}$$

Răspuns corect: E

6. (*Calorimetrie*) Se amestecă $m_1 = 200$ g de apă la $t_1 = 80^\circ\text{C}$ cu $m_2 = 300$ g de apă la $t_2 = 20^\circ\text{C}$, într-un calorimetru ideal (fără pierderi). Temperatura de echilibru este:

- A) 50°C C) 44°C E) 44 K
 B) 40°C D) 60°C F) 48°C

Rezolvare

Într-un sistem izolat, căldura cedată de apa caldă este egală cu căldura primită de apa rece:

$$|Q_{\text{cedat}}| = Q_{\text{primit}} \implies m_1 c(t_1 - \theta) = m_2 c(\theta - t_2)$$

Simplificăm căldura specifică c și obținem direct temperatura de echilibru θ :

$$\theta = \frac{m_1 t_1 + m_2 t_2}{m_1 + m_2} = \frac{200 \cdot 80 + 300 \cdot 20}{500} = \frac{16000 + 6000}{500} = \frac{22000}{500} = 44^\circ\text{C}$$

Răspuns corect: C

7. (*Mașină termică Carnot*) O mașină termică ideală (Carnot) funcționează între temperaturile $t_1 = 227^\circ\text{C}$ și $t_2 = 27^\circ\text{C}$. Randamentul mașinii este:

- A) 88 % C) 50 % E) 60 %
 B) 11,9 % D) 40 % F) 100 %

Rezolvare

Randamentul mașinii Carnot se calculează exclusiv folosind temperaturile absolute (în Kelvin). Aceasta este „capcana” clasică a problemei.

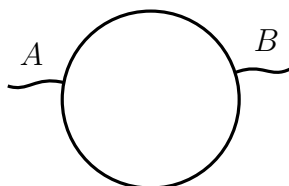
- $T_1 = 227 + 273 = 500\text{ K}$
- $T_2 = 27 + 273 = 300\text{ K}$

$$\eta = 1 - \frac{T_2}{T_1} = 1 - \frac{300}{500} = 1 - 0,6 = 0,4 = 40\%$$

Răspuns corect: D

Electricitate

8. (*Inel conductor – putere minimă*) Un fir conductor omogen este îndoit sub forma unui inel închis. Inelul este alimentat la o sursă de tensiune constantă $U = 6\text{ V}$ prin intermediul a două borne, A și B , situate pe circumferința sa. Puterea minimă disipată de circuit este $P_{\min} = 36\text{ W}$. Să se determine rezistența electrică totală a inelului conductor.



A) $1 \, \Omega$ C) $4 \, \Omega$ E) $0,75 \, \Omega$ B) $2 \, \Omega$ D) $10 \, \Omega$ F) $6 \, \Omega$ **Rezolvare**

Punctele A și B împart inelul în două porțiuni conductoare conectate în paralel. Fie R rezistența totală a inelului, iar R_1 și R_2 rezistențele celor două arce, astfel încât $R_1 + R_2 = R$.

Rezistența echivalentă a sistemului paralel este:

$$R_e = \frac{R_1 R_2}{R_1 + R_2} = \frac{R_1(R - R_1)}{R}$$

Puterea disipată în circuit este $P = \frac{U^2}{R_e}$. Pentru ca puterea disipată să fie minimă ($P_{\min}$), rezistența echivalentă a circuitului trebuie să fie maximă.

Expresia $R_1(R - R_1)$ este un polinom de gradul doi care își atinge maximumul atunci când $R_1 = \frac{R}{2}$ (adică bornele sunt situate diametral opus).

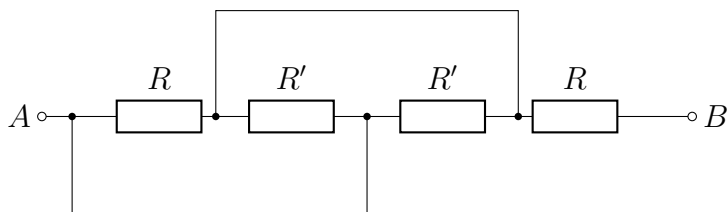
$$R_{e,\max} = \frac{\frac{R}{2} \cdot \frac{R}{2}}{R} = \frac{R}{4}$$

Scriem relația pentru putere:

$$P_{\min} = \frac{U^2}{R_{e,\max}} = \frac{4U^2}{R} \implies R = \frac{4U^2}{P_{\min}} = \frac{4 \cdot 36}{36} = 4 \, \Omega$$

Răspuns corect: C

9. (*Rezistență echivalentă*) Se consideră circuitul din figura alăturată. Să se calculeze rezistența echivalentă între punctele A și B , știind că $R' = 2R$.



A) $\frac{3R}{2}$

C) $\frac{2R}{3}$

E) $\frac{R}{3}$

B) R

D) $6R$

F) $\frac{5R}{2}$

Rezolvare

Pentru a simplifica circuitul, urmărim conductoarele ideale (punțile) care scurtcircuitază anumite noduri, stabilind potențiale egale:

1. Puntea de jos conectează nodul de la intrarea primului rezistor (nodul A) direct între al doilea și al treilea rezistor. Acest nod capătă potențialul A .
2. Puntea de sus conectează nodul situat între primul și al doilea rezistor direct cu nodul situat între al treilea și al patrulea rezistor. Să numim acest nod intermediar comun C .

Reconstruind schema între nodurile A și C , observăm că primele trei rezistențe (R , R' și R') sunt conectate toate cu un capăt în A și celălalt în C . Deci sunt în paralel.

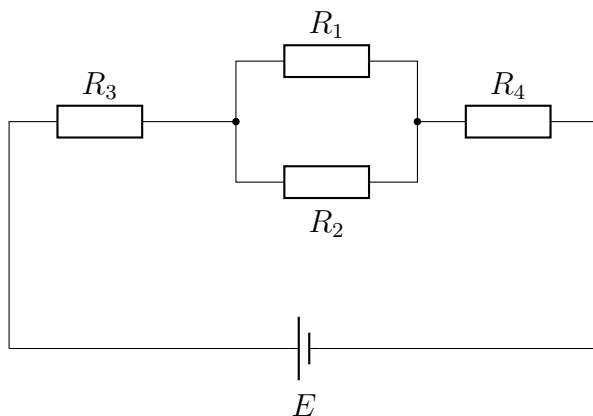
$$R_p = \left(\frac{1}{R} + \frac{1}{R'} + \frac{1}{R'} \right)^{-1} = \left(\frac{1}{R} + \frac{1}{2R} + \frac{1}{2R} \right)^{-1} = \left(\frac{2}{R} \right)^{-1} = \frac{R}{2}$$

Rezistorul al patrulea (de rezistență R) este conectat între nodul C și ieșirea B . Prin urmare, acesta este în serie cu gruparea paralelă:

$$R_e = R_p + R = \frac{R}{2} + R = \frac{3R}{2}$$

Răspuns corect: A

10. (*Curent maxim*) În circuitul alăturat se consideră $R_1 = 9 \, \Omega$, $R_4 = 4 \, \Omega$ și $E = 120 \, \text{V}$. Să se calculeze valorile rezistențelor $R_2 = R_3$ astfel încât intensitatea curentului prin R_1 să fie maximă.

A) 2Ω C) 13Ω E) 3Ω B) 1Ω D) 7Ω F) 6Ω **Rezolvare**

Notăm $R_2 = R_3 = x$. Analizând circuitul, ramura centrală conține rezistențele R_1 și R_2 legate în paralel, având o rezistență echivalentă:

$$R_p = \frac{R_1 x}{R_1 + x} = \frac{9x}{9 + x}$$

Această grupare paralelă este inseriată cu R_3 și cu R_4 , deci rezistența totală a circuitului este:

$$R_e = x + \frac{9x}{9 + x} + 4$$

Curentul total generat de sursă este $I = \frac{E}{R_e}$.

Curentul care trece prin ramura superioară (R_1) se află folosind regula divizorului de curent:

$$I_1(x) = I \frac{R_2}{R_1 + R_2} = \left(\frac{E}{x + \frac{9x}{9+x} + 4} \right) \cdot \frac{x}{9 + x}$$

$$I_1(x) = \frac{Ex}{(x+4)(x+9)+9x} = \frac{Ex}{x^2+13x+36+9x} = \frac{Ex}{x^2+22x+36}$$

Pentru a găsi maximul funcției $I_1(x)$, rescriem ecuația coborând pe x la numitor:

$$I_1(x) = \frac{E}{x + \frac{36}{x} + 22}$$

Intensitatea I_1 este maximă atunci când numitorul este minim. Din inegalitatea mediilor (sau derivând și egalând cu zero), suma $x + \frac{36}{x}$ este minimă când termenii sunt egali:

$$x = \frac{36}{x} \implies x^2 = 36 \implies x = 6 \Omega$$

Răspuns corect: F